The Review–Execution Gap in Agentic Software: Approval Scope and Recovery

Liang Hanghao[1], Xuan Wentao[1], Chen Qile[1], Peng Peng[1*]

[1]College of Computer Science and Electronic Engineering, Hunan University, Changsha, 410082, China.

*Corresponding author(s). E-mail(s): hnu16pp@hnu.edu.cn;

Abstract

Human review constrains what agentic workflows may execute, yet agents can regenerate or resume proposed updates after approval. A correct final value alone cannot establish that the executed candidate was covered by the review. We investigate how an application's declared approval scope governs admission at a durable execution boundary and how agents continue after an instance mismatch. We develop a four-role correction contract linking the machine proposal, authorized correction, reviewed candidate, and persisted transition. For context- and instance-bound admission, we hold the shared authority, freshness, atomicity, and provenance obligations fixed; controlled comparator cases additionally examine content binding and external-reference changes. Relational and event-journal implementations and bounded formal checks assess the contract's persistence obligations. A configured ToolGate adaptation tests implementability, while pinned SDK probes separately examine reviewed-argument consumption and continuation routes. In a reduced agent host with a deterministic reviewer, 32 delivered identity-only substitution pairs span 16 task definitions. Context binding admits every substitute, and 30 context continuations complete. Instance binding rejects every substitution; all 32 instance-bound continuations recover, with 24 reusing the reviewed candidate and eight obtaining fresh authorization. Archived states and model inputs establish that reuse remained available and visible in all 32 episodes. These continuations incur additional tool calls, without an established task-completion advantage. Approval scope therefore defines admissible reuse, while agents' subsequent choices determine recovery routes and their interaction costs. The authorization boundary and the continuation it enables must be designed and evaluated together.

Keywords: Agentic software, Human review, Runtime enforcement, Authorization, Provenance, Empirical evaluation

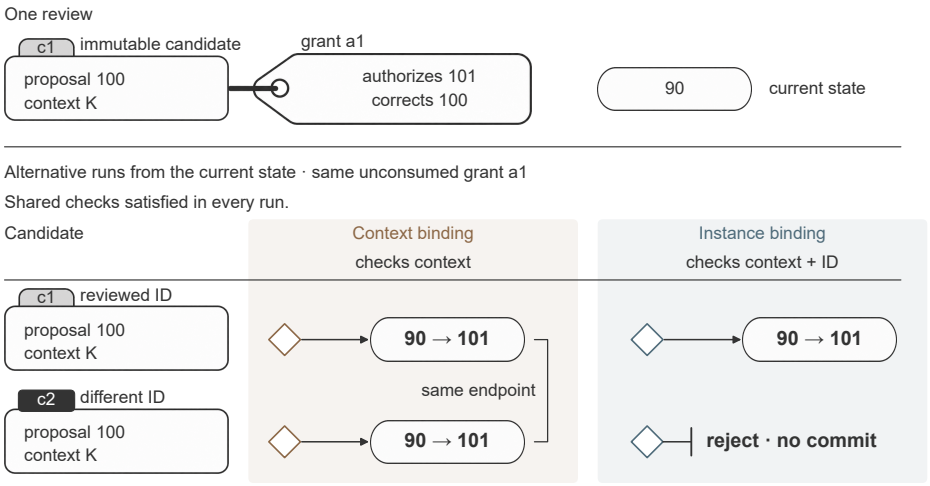


Fig. 1 Equal endpoints do not identify the reviewed instance. Candidate c1 proposes 100; grant a1 authorizes the correction 101 for c1. Candidate c2 retains the same context K with a different identifier. Alternative paths start from state 90 with a1 unconsumed and all shared checks satisfied: context binding admits either candidate, whereas instance binding admits only c1. Diamonds are checks, capsules are states/transitions, and the stop bar means no commit. A matching new grant may authorize c2. The four roles are proposal 100, correction 101, reviewed c1 and executed transition 90 → 101. Conceptual construction; AI-generated object glyphs are disclosed below

1 Introduction

Human approval depends on understanding a proposed change (Bacchelli and Bird 2013) and determining whether that decision still applies at execution. Established workflows make this applicability configurable: Gerrit controls whether review votes copy across patch sets and which submit requirements apply (Gerrit Project 2026a,b), while GitLab distinguishes deployment approval from merge-request approval reset (GitLab 2026a,b). Each approval object—revision, change or deployment instance—determines which replacements remain covered.

In agentic software, a language-model controller selects tool actions from execution feedback (Wang et al. 2026b), while approval middleware can pause, edit, reject and resume a call (LangChain 2026). A controller may regenerate or retrieve a candidate after review. Even if its content and intended effect are unchanged, the application must decide whether the earlier business grant covers that candidate.

Consider an agent-generated release-change record c1 that proposes replacing 90 with 100. A reviewer authorizes the correction 101. Before persistence the workflow produces c2 with the same proposal, evidence, target and expected predecessor. Executing c2 produces the same final value as executing c1, but whether the earlier approval authorizes c2 depends on the application's approval scope: a context- or content-scoped policy may permit the replacement, while an instance-scoped policy requires the reviewed record or a new matching authorization. The final state alone cannot distinguish these authorization histories. We call the unresolved relationship between the reviewed object and the executed operation the **review–execution gap**.

The gap has two connected parts: enforcing approval scope together with value correctness, predecessor freshness and provenance, and enabling continuation after rejection. An agent may reuse the reviewed candidate, obtain new authorization or stop when no valid route remains. These routes have different costs even when they reach the same business state. GitLab's deployment-specific approvals illustrate an operational instance rule (GitLab 2026a); the interface alone does not determine an agent's continuation or its cost.

Recent work has examined reconstruction-aware authorization and transactional approval enforcement (Zhu et al. 2026; Bindschaedler et al. 2026), as well as post-review substitution and approval–execution binding failures (Kumar 2026; Wang 2026). However, it remains unclear how a declared approval scope jointly determines which persisted candidates are admissible and how an agent continues after rejection. We study these outcomes under shared correction, freshness, provenance and atomicity obligations.

We ask three questions. **RQ1**: which candidate replacements can legitimately reuse an existing approval under context-, content- and instance-based policies? **RQ2**: how can the selected approval relationship be checked together with authorized corrections, current predecessors, atomic transitions and field-level provenance? **RQ3**: after an instance mismatch, which legal recovery routes do agents take, and what do those routes cost?

We make three connected contributions.

1. **A unified comparison of correction and approval scope.** We formulate a four-role contract connecting the machine proposal, the authorized correction, the reviewed candidate and the persisted transition. Holding value, freshness, provenance and atomicity obligations fixed, we characterize how approval scope changes admissibility, and nine constructed cases separate omitted-field changes, equivalent re-materialization and mutable external references, identifying which protections follow from the binding rule and which require a separate consumption check.
2. **Cross-layer validation of review-to-execution continuity.** Relational and event-journal stores assess the durable relations, checked by an independent oracle and bounded formal analysis. A ToolGate adaptation tests whether an existing gate can carry the obligations through its extension interface. Three pinned SDKs then examine what a resumed call executes: a matching native call identifier need not preserve the reviewed arguments, although these probes do not test contract enforcement.
3. **An empirical account of recovery under explicit approval rules.** At controlled handoffs in a reduced host with a deterministic reviewer, 32 delivered identity-only substitutions from 16 task definitions are admitted under context binding and rejected under instance binding. All 32 rejected continuations recover, but agents choose different legal routes: 24 reuse the reviewed candidate and eight obtain fresh authorization. Archival reconstruction establishes that the reviewed candidate and its grant were available and visible in the saved model input in all 32 episodes, including the eight that reauthorized. We separate recovery from

later confirmation and renewed authorization requests and report their call and elapsed-time costs.

Table 1 maps each question to its evidence. The agent comparison holds retained content fixed ($K = R$); it measures the behavioral cost of an instance requirement.

2 Related Work

2.1 What existing mechanisms bind

Table 2 compares binding objects, mismatch handling and check points. Configurable code-review approval (Gerrit Project 2026a; GitLab 2026b) motivates our comparison for corrected agent state and subsequent continuation.

ReSA and Guarded Commits are the closest comparisons. ReSA derives repair obligations from consumed versions and grant scope, so that content equality does not establish preservation (Zhu et al. 2026); Guarded Commits checks pinned policy and path coverage at commit and binds decisions to artifact hashes (Bindschaedler et al. 2026). Both begin from a correction or execution path and ask what authority it needs. Our comparison fixes those correction and persistence obligations, varies what the approval is bound to, and observes continuation when a candidate fails the binding.

Loopjacking and Approval Laundering document the same relation failing in released software: incomplete approval representations, post-approval state substitution, and divergence between approved invocations, dispatch and effects (Kumar 2026; Wang 2026). We use their archived version-1 preprints, as for ReSA. Our controlled

Table 1 Research questions, evidence and the claim each layer supports. Controlled conditions, trajectories and historical records are distinct units.

Question	Evidence	Supported claim
RQ1	Local specification and nine constructed cases	Distinguish approval scope under stated storage and comparator assumptions (Sections 3.3, 5.3).
RQ2	165 controlled cases per mechanism (context journal; exact journal and relational reference); 24 scripted public routes per implementation	Validate durable relations and framework extension feasibility (Section 5).
RQ2	16 native interrupt conditions; 10 scripted integration conditions; 504 reviewed-argument cells; 54 route-ablation cells; 16 substitution cells	Distinguish native resume routing and application grant enforcement (Section 5.3; Online Resource 2, A14, A17).
RQ3	32 delivered identity-handoff pairs; archived state and visible-input reconstruction	Observe recovery routes and agent-side costs across 16 task definitions (Section 6.3).
RQ3	12 exploratory feedback pairs and 24 rule trajectories	Test continuation under three permission states; compare path labels with the same facts (Section 6.4).

Table 2 Binding, mismatch handling and evidence in the cited sources. The two attack analyses describe failure paths; their check points are observation points.

Mechanism	Binding object	Mismatch handling	Check point	Evaluation form
LangChain HITL (LangChain 2026)	Interrupted tool call	Reviewer may edit/reject, then resume	Before tool execution	Interface documentation
ReSA (Zhu et al. 2026)	Exact/scoped grant; consumed versions	Derive repair obligations	Reconstruction to local sink	Authored graphs; installed-control checks
Guarded Commits (Bindschaedler et al. 2026)	Artifact, policy; edit hashes	Fresh authorization or review	Commit	Public-trace replay; synthetic DAGs
Cordon (Chen et al. 2026)	Task, lineage, staged effects	Abort, rollback, compensation	Commit or effect release	Agent workflows; scripted rollback/resume
Continuity Kernel (He and Yu 2026)	Proposal ID; sealed successor	Reject, quarantine or defer; new-ID retry	Atomic activation	Bounded model exploration
Commit-time authorization (Santos-Grueiro 2026)	Witness, target, eligibility	Abort or refresh/rebind	Durable-effect boundary	Agent perturbations; replay; limited enforcement
ToolGate (Liu et al. 2026)	Symbolic state; tool contracts	Filter call; discard invalid result	Tool invocation; state update	ToolBench; MCP-Universe
Gerrit / GitLab (Gerrit Project 2026a,b; GitLab 2026b)	Patch set / merge request	Copy or reset votes by policy	New revision; submit/merge	Product documentation
OpenTofu (OpenTofu 2026)	Saved plan decisions	Plan mismatch: not assessed in cited guide	Plan application	Command documentation
Microsoft Agent Framework (Microsoft 2026)	Pending request in session; standing tool/argument rules	Unbound response ignored; changed approved arguments fail closed	Session approval middleware	Vendor documentation; opt-out and standing rules
Loopjacking (Kumar 2026)	Analyzed: call representation and post-review state	Substitution attack; deny-update control	Review, update, resume	Released Agno; configured LangGraph tests
Approval Laundering (Wang 2026)	Analyzed: grant, dispatch, effects	Policy divergence; token-replay check	Dispatch; effect sinks	Claude Code probes under research policies
This work	Compared: context, content and instance	Reuse reviewed instance or reauthorize	Atomic successor write	Formal/store checks; paired agent continuation

intervention holds correction and retained content equal, so the declared scope is the only variable, and the outcome is the agent's recovery route.

Other mechanisms bound the same problem from different sides: Continuity Kernel seals proposal identity through atomic activation (He and Yu 2026), commit-time authorization separates endpoint success from authority at durability (Santos-Grueiro 2026), and Cordon studies workflow execution and approval costs (Chen et al. 2026). ACG derives a minimal repair frontier for a fixed action from versioned authorization dependencies and revalidates at execution (Wang et al. 2026a). These are foundations and alternative responses; we do not propose a planner, but measure the routes agents choose when the reviewed instance remains available. ToolGate provides the external extension interfaces tested in Section 5.2.

Microsoft Agent Framework documents the same axis in released tooling: a local approval response must match a same-session pending request, unbound responses are ignored, standing tool- or argument-scoped rules are not bound to the tool instance, and middleware that changes approved arguments must revalidate or fail closed (Microsoft 2026). Framework fuzzing uses specification-driven inputs and an agentic oracle (Long et al. 2026). Our native checks instead compare reviewed objects with executed effects, treating runtime identifiers as observations rather than recovered mappings.

2.2 Foundations and the correction relation

Transactions and concurrency control supply atomicity and predecessor validation (Gray 1981; Kung and Robinson 1981). Provenance covers derivations, edits and copies (Buneman et al. 2001; Cheney et al. 2009; Buneman et al. 2006), transaction reenactment (Arab et al. 2018) and entity/activity/agent descriptions (World Wide Web Consortium 2013). Interactive repair separates machine suggestions from human judgments (Yakout et al. 2011; He et al. 2016). Our contract joins these concerns in the installed successor's authorized correction, reviewed-candidate link and field sources.

Integrity and capability systems connect principals, permitted operations and protected objects (Clark and Wilson 1987; Hardy 1988; Miller 2006). Runtime verification and monitoring address execution conformance and enforceable policies (Leucker and Schallhart 2009; Schneider 2000); information-flow control constrains propagation (Sabelfeld and Myers 2003), and accountability supports reconstructible, attributable execution (Haeberlen et al. 2007).

These foundations supply the contract's individual checks. Holding correction, freshness and persistence fixed leaves the binding relation as the only variable. The admission difference then follows from the definitions, whereas the choice among available recovery routes, renewed authorization requests and subsequent confirmation does not. Online Resource 2, A15 maps each obligation to its foundations.

3 A Contract for Corrected Execution

A comparison of approval scopes is interpretable only if every policy is judged against the same correction, freshness, provenance and atomicity obligations. The contract below fixes those obligations so that binding is the only variable.

3.1 Four roles

Figure 1 illustrates the four roles: proposal 100, authorized correction 101, reviewed candidate c1 and the transition from 90 to 101.

A persisted candidate is

$$c = \langle \mathrm{id}, r, f, e, v_{\mathrm{exp}}, x_c, p \rangle$$

where id names a content-immutable persisted candidate record, (r, f) the record and field, e the evidence and locator, v_{exp} the expected predecessor, x_c the proposal, and p the producer. An authorization $a = \langle c_r, x_a, q \rangle$ records the reviewed candidate, explicitly authorized value and authorized principal. Execution submits candidate

c_s and value x, producing a persistent transition. Online Resource 2, A9 types its candidate and authorization references and defines satisfaction over the pre-state, request and post-state. These elements define the four roles: the **machine proposal**, the **authorized correction**, the **reviewed candidate** and the **executed transition**. The proposal remains in the record when correction makes $x_c \neq_{\text{json}} x_a$; the successor must contain x_a.

Retained context $K(c)$ comprises record and field, evidence identity and locator, expected predecessor, proposal and producer. Candidates are *context-equivalent* when they agree on K, and *content-equivalent* when they agree on declared fields $R \supseteq K$, under the specified comparator. *Same-semantics* means the same approved effect under the scenario oracle. An *identity-only substitution* preserves the declared tuple's non-identifier components while changing its identifier. Instance-creation and outer envelope metadata lie outside this tuple. Value comparisons use canonical JSON equality.

3.2 Admission and persistence

Define context and identity agreement by

$$K_{\text{eq}} \Leftrightarrow K(c_s) = K(c_r), I_{\text{eq}} \Leftrightarrow \text{id}(c_s) = \text{id}(c_r)$$

Let G collect the shared checks: valid authority and grant, context/evidence integrity, authorized-value equality, current predecessor, complete successor, source relations and transaction constraints. The integrity component of G concerns whether the submitted candidate is internally consistent with its evidence; it is separate from K_{eq}, which compares the submitted and reviewed contexts. The two policies are

$$P_{\text{context}} : \text{Admit} = G \wedge K_{\text{eq}}$$
$$P_{\text{bound}} : \text{Admit} = G \wedge K_{\text{eq}} \wedge I_{\text{eq}}$$

The contract assumes that a trusted review channel supplies an authenticated reviewer decision linked to the displayed candidate; it does not assess informed consent or decision quality. Both policies retain the reviewed identifier in the authorization; they differ only in the execution rule. Context binding treats the retained information as the approval object; instance binding attaches responsibility to the reviewed artifact. A same-key retry retrieves the existing result, whereas re-preview can materialize a new candidate requiring a matching grant.

The contract combines six obligations: **P1**, the selected binding relation (K_{eq} for context and $K_{\text{eq}} \wedge I_{\text{eq}}$ for bound); **P2**, internal context and evidence integrity; **P3**, principal authority and the authorized value; **P4**, predecessor freshness; **P5**, one complete atomic successor; and **P6**, field-source relations. Online Resource 2, A9 maps these labels to the archived Alloy assertions. Changed fields point to their authorized transitions; unchanged fields retain their exact prior sources. The successor, transitions, sources and grant consumption commit atomically. Direct-write exclusion (P0) places all authoritative mutation behind this boundary. After initial coverage, concrete admission requires a nonempty value change; same-value re-attribution uses a separate review-event path (Online Resource 1, S2).

Trusted candidate storage is a separate prerequisite: a candidate identifier must resolve to one content-immutable record, and the executor must consume that record.

Candidate creation does not confer permission to overwrite an existing candidate or its review linkage. P0 excludes bypass writes to authoritative state; it does not by itself establish this storage prerequisite. External referents require the separate consumption check in Section 5.3.

3.3 What endpoint correctness cannot establish

The exact policy difference is the request-state set

$$S = \{r : G(r) \wedge K_{\mathrm{eq}}(r) \wedge \neg I_{\mathrm{eq}}(r)\}$$

so bound admission refines context admission. This inclusion follows directly from the policy definitions, not from measurement. Its methodological role is to separate what a policy definition alone determines from the behavior that follows when a request is excluded; Section 6 measures that behavior. The following witness identifies information that endpoint checks cannot recover.

Endpoint indistinguishability. *Equal predecessor states and equal final values can carry different review continuity.* Take independent copies of the same pre-admission state and unconsumed grant. In one copy, context binding admits (c_s, x_a); in the other, instance binding admits (c_r, x_a). When the candidates differ only in identity, both successors have the same field-wise values and copied sources, but only the second transition uses the reviewed instance. The archived fixed-registry witness isolates this difference while holding the other observations fixed. Because the two executions also agree on retained context, a check that reads only final values, predecessor freshness and retained context assigns both the same verdict; continuity must be observed through the candidate–authorization binding itself.

With authorization parameters fixed, five information classes suffice for the declared local classification: candidate–authorization binding, role-labeled values, predecessor discriminators, successor structure and field sources. Online Resource 2, A9 defines the classification and gives witness pairs establishing irredundancy; Online Resource 1, S2 supplies executable checks.

Gate soundness is conditional on direct-write exclusion, a trusted gate and adapter, authority checked at commit entry, consistent equality, transaction-scoped reload and predecessor validation, and atomic installation of the checked relations. Under these premises, the observed bound-commit tuple satisfies the local contract defined in A9 (P1–P5 and the declared local source checks). Full trace soundness (P6) additionally requires a trace-complete predecessor, immutable valid references, validated new chain links and uniqueness of the successor's transitions; this extension is supported by bounded preservation checks and audits rather than by a general proof. Conversely, a nonempty change that satisfies the remaining contract conditions is accepted when the gate observes the authoritative state and its grant and key remain usable; the detailed acceptance conditions are in A9.

3.4 Continuation after rejection

For an identity-only mismatch, the agent can reuse the reviewed candidate under its still-valid grant or obtain/use a matching authorization for another candidate. Changed

evidence or a stale predecessor requires a new candidate reflecting current state and matching authorization. A consumed grant with a prior commit instead calls for receipt retrieval or state confirmation. Each new admission must satisfy the remaining checks.

A rejection thus identifies the failed relationship, and the continuation shows whether an agent can restore it. Section 6 measures those subsequent actions and their cost.

4 A Design Space for Binding Review to Execution

Approval scope expresses an application responsibility rule: instance binding preserves the reviewed artifact; context binding permits interchangeable candidates. Under immutable records and shared checks, stronger binding narrows admission, while reauthorization changes the continuation branch. API-specific comparison semantics remain essential (Lodderstedt et al. 2023). The inclusion is definitional; the empirical question is the continuation cost (Section 6).

Three axes organize this choice (Figure 2). The first is the binding object: no gate, value, context, content or instance. Under trusted lookup of content-immutable records, identity equality implies content equality and hence context equality. Writing $P_{\text{value}} = G$, and defining content binding on fields $R \supseteq K$, gives $P_{\text{bound}} \subseteq P_{\text{content}} \subseteq P_{\text{context}} \subseteq P_{\text{value}}$ under shared checks. Thus K_{eq} is redundant under instance binding. The adapter must preserve the resolved immutable record to use: the trusted-preservation case of ReSA (Zhu et al. 2026), with external documents checked separately at consumption. The contract and queued-handoff agent experiment (family G, Section 6.1) use $K = R$ over the declared non-identifier fields, excluding outer host metadata; the levels separate when retained context omits a decision-relevant field (Section 5.3). Thus the agent study does not estimate a separate content-versus-context effect.

The second axis is mismatch handling: adoption with a substitution record, unexplained denial, actionable rejection or routing to reauthorization. The tested context policy adopts the substitute and records its identifier. Reauthorization is a continuation branch, not another binding level: it retains the instance decision at admission but provides a matching new grant. The third axis is the check's anchor and timing: request, server-held review record or persisted constraint, checked at review, handoff, commit or grant consumption.

Online Resource 2, A10 maps responsibility rules, compared fields, trusted persistence and continuation to implementation choices. Section 5.3 compares three binding rules deterministically; Section 6 tests agents under context binding with recorded substitution and instance binding with a continuation channel. The remaining combinations are design options.

GitLab protected-environment deployments must satisfy every configured approval rule. Its deployment-specific approval endpoint accepts `approved` or `rejected`, and each user may approve a deployment only once across approver groups (GitLab 2026a). Redeployment of identical content must satisfy the rules again; merge-request settings separately control vote reset on new commits (GitLab 2026b). These interfaces establish an operational policy choice, not the prevalence of inappropriate content-scoped approval or unauthorized deployment.

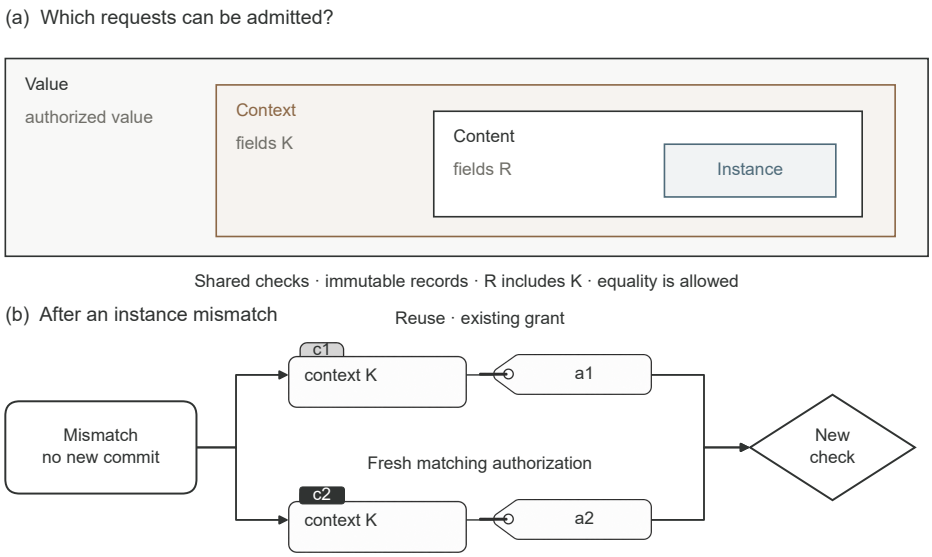


Fig. 2 Binding and recovery are separate design choices. Under shared checks and content-immutable candidate records, if the compared content fields R include the retained context fields K, the admissible request sets satisfy $A_{\text{instance}} \subseteq A_{\text{content}} \subseteq A_{\text{context}} \subseteq A_{\text{value}}$. These inclusions need not be strict; context and content binding coincide when $K = R$. Following an instance-mismatch rejection, the system may offer reuse of the reviewed candidate under the existing authorization or obtain matching authorization for another candidate. Reauthorization is a recovery branch, not another binding level. Each continuation must pass a new admission check; the schematic does not guarantee that an available route will be taken. Object glyphs were AI-generated; see the AI-assisted preparation declaration

5 Realization and External Comparison

The relational and event-journal implementations check stored state; a ToolGate adaptation tests whether an existing gate can carry the obligations through its extension interface; and three pinned SDKs determine which reviewed arguments reach execution.

5.1 Execution gate and independent checks

The contract runs in a relational store and an event journal. The relational adapter reloads state, evidence and authorization within the transaction, validates the principal and unused identities, compares-and-swaps the predecessor, and installs the complete successor, transitions and exact sources. The journal records the same relations under context and exact configurations differing only in instance equality. An independent database oracle reads persisted pre/post relations; a source verifier follows field links through transitions, authorizations, candidates, evidence and predecessors.

The controlled suite runs 165 cases per mechanism. The context journal accepts 60 and rejects 105; the exact journal and relational reference each accept 45 and reject 120. The 15 differences are same-context substitutions. The two exact implementations also agree on provenance-query answers. Request/display probes and the case mapping appear in Online Resource 2, A6.

Table 3 Configured ToolGate (T) versus the contract backend (C). Development controls and public workflow routes are separate units; decisions and recovery are scripted.

Check	T	C
Development controls satisfied	8/8	8/8
Public: initial authorized business commits	12/12	12/12
Public: invalid requests rejected without a business write	12/12	12/12
Public: fixed legal recovery commits	12/12	12/12
Public: complete reference-world matches	24/24	24/24
Public: frozen strict PASS/FAIL	23/1	24/0

Six public workflows crossed with four controlled scenarios give 24 routes per implementation. The retained strict failure is two initialization log lines violating an empty-output rule. Post-hoc inspection confirms that route's business relations and final reference world; the frozen FAIL is unchanged. Reference-world agreement is not an official benchmark success rate or evidence of autonomous recovery.

Bounded formal checks assess the encoded contract. Alloy/Kodkod checks (Jackson 2002; Torlak and Jackson 2007) cover admissions, conjunct removal, source attacks and trace preservation: 72 command outcomes match the manifests, with no assertion counterexamples in the encoded finite scopes. The independent mapping has nine intended satisfiable projections and twenty unsatisfiable mutants (Online Resource 1, S2).

A separate historical audit reconstructs 84 machine-origin correction chains from 1,260 hosted databases. It establishes that the four roles can be recovered from retained persistence records. Per-association and trace counts are reported in Online Resource 2, A5; unchanged-field copying is exercised in the controlled multi-field cases, not these single-field chains (A6).

5.2 ToolGate implementability case study

ToolGate (Liu et al. 2026) supplies symbolic state and application pre/postconditions. Its normal callbacks express the binding, value, predecessor and source obligations; an author-written SQLite bridge supplies the record schema and durable commit boundary. This tests implementability through extension interfaces, not native default admission.

Execution has four stages: callbacks check the request; the native tool drafts changes in a cloned world; postconditions validate the draft; the bridge installs the successor and its relations in SQLite. Persisted state is then projected back into the public world for the reference suffix (Online Resource 2, A8).

Both implementations pass eight development controls and the shared business and scripted-recovery checks on 24 routes: six pinned tau2-bench Retail workflows (Barres et al. 2026) crossed with four scenarios (Table 3). ToolGate's frozen strict failure concerns initialization output, separately retained from business-state scores. Online Resource 2, A8 documents these scripted implementability tests.

5.3 Binding scope and native defaults

Deterministic probes examine native resume routing, native-to-store composition and binding scope. None calls a model. Online Resource 2, A11 and A14 give the separate protocols, saved runs and reproduction commands.

Native human-in-the-loop defaults. Unmodified LangChain middleware (LangChain 2026) and LangGraph interrupts (versions 1.2.10 and 1.0.8) are tested in author-written graphs, executors and recorders. Sixteen conditions record 18 SQLite effects. Across two turns on the same graph, checkpointer and thread, the old interrupt identifier leaves the new review pending without writing; the current identifier permits one write. A repeated affirmative input at the new interrupt is fresh approval. An explicit edit from 100 to 999 persists 999, preserving the original proposal in checkpoint history; malformed middleware decisions fail without writing. These routes show approval of the pending interrupt, not reuse of an earlier approval for a later request. Same-identifier pending-call replacement and Loopjacking's Agent Server configurations are outside this probe (Kumar 2026).

Reviewed-argument delivery across a restart. Three separately frozen checks in Online Resource 2, A17 test Agno 3.1.10, LangChain 1.4.7 with LangGraph 1.2.16, and Pydantic AI 2.60.0 (Table 4). Of 504 constructed cells supplied with complete reviewed per-call objects, 444 match the expected effect multiset. All 84 cells per lifecycle match for LangChain and Pydantic AI, as do Agno's 84 fresh-process cells. Agno's original in-memory run-object route executes another pending call's reviewed arguments in 60 of 72 multi-call cells, reproducing documented issue #8546.

In a 54-cell ablation, the run-identifier route matches all 18 cells both in process and after a clean restart; the original-object route matches six single-call cells and mismatches all twelve multi-call cells. This implicates the continuation route and its state, while entrypoint and object aliasing change together.

A separate 16-cell check keeps the native call identifier fixed: approval without resupplying the reviewed object executes substituted arguments in all four cells modifying the supplied message history. Native override executes the reviewed object in all eight override cells, including six substitutions and two unchanged controls. Client-supplied history is a documented application-maintained trust boundary (Pydantic Services Inc. 2026); this result does not identify a framework defect.

Native-to-store composition. A further scripted graph connects LangGraph 1.0.8 review/resume to the unchanged relational gate and recovery within one in-process execution. Ten conditions cover correction, denial, identity substitution with reuse or fresh review, wrong values, stale predecessors, two rollback points and native/application replay. Both frozen executions satisfy all ten conditions. Per execution, eight authorized corrections commit, while six invalid or rolled-back attempts leave no partial transactional rows; the stale case preserves an independent update. An old interrupt identifier cannot resume the replacement's fresh review. Direct SQL readback checks the candidate, authorization, transition and unchanged-field source relations. This establishes deterministic composition for the tested paths. The trusted adapter registers candidates and creates pre-commit grant handles; the relational gate persists accepted authorization and transition relations atomically. Grant handles and checkpoints remain in process, and decisions are scripted (Online Resource 2, A14).

Table 4 Native-SDK checks in Online Resource 2, A17. Constructed cells; the three checks are separate frozen collections scored by the same effect oracle

Check	Cells	Controlled contrast	Observed outcome
Reviewed-argument delivery	504	Three SDKs × two lifecycles × seven call scenarios × six review edits × two payload families	444 exact matches; the Agno in-process multi-call cells share the later call's reviewed values
Continuation route	54	Original in-memory run object; run-identifier route in the same process; run-identifier route after a clean restart	Run-identifier route matches 18/18 in both lifecycles; original-object route matches 6/18
Reviewed-instance substitution	16	Identifier-only approval versus override with the reviewed object; substitute the supplied history, the deferred request object or both	Identifier-only approval executes substituted history arguments (4/4); override executes the reviewed object (8/8)

Binding scope. Nine constructed cases cross three binding comparators and a diagnostic variant with two execution modes, giving 72 routes (54 principal, 18 diagnostic). All arms apply the same request-value guard and differ only in the compared fields (Table 5). In one case the reviewed document changes after review while the candidate stays identical, and a matched control leaves it unchanged. An unchecked executor reads the live document; a verifying executor checks the reviewed digest and consumes the same captured bytes, stopping on mismatch. An independent scorer reads only the persisted effects and compares them with the approved effects saved before execution; it detects deliberate deletion, addition and alteration in copies of each sink.

This experiment uses an extended candidate schema: K contains the target record, operation, unit, source-document digest and proposal; R adds effective date and cost center. A note enters both projections only when declared semantic. Monetary values are normalized to two decimal places. These comparators and the common executor test binding scope, not the full shared gate G. In contrast, the contract's declared tuple and the queued-handoff agent experiment use $K = R$ with canonical JSON equality.

Context binding admits three changes to the omitted effective-date or cost-center fields; checking the external document's digest does not cover those fields. Content binding rejects the changes through field equality. Instance binding rejects them because each immutable-record change creates a new identifier, avoiding field enumeration in the reuse key. It also rejects three same-semantics re-materializations admitted by content binding: an identical new instance, representation change and display-only note. Section 6 measures recovery from identity-only substitutions. The changed-document case passes all four bindings because the candidate is unchanged: the unchecked executor writes the changed amount, the verifying executor writes nothing, and both write the reviewed amount in the unchanged control. Referent protection comes from the executor's consumption check.

Table 5 Binding-gate decisions over nine constructed cases. A: admit; R: reject. Each decision is identical under unchecked and verified execution (72 routes); entries do not report writes. All comparators share the request-value check.

Case	Context	Content	Instance	Raw content (diagnostic)
C1: Identical new instance	A	A	R	A
C2: Effective-date change[a]	A	R	R	R
C2: Cost-center change[a]	A	R	R	R
C2: Date change + render noise[a]	A	R	R	R
C3: Changed external document[b]	A	A	A	A
C4: Number/string representation	A	A	R	R
C4: Display-only note	A	A	R	A
C4: Semantic note	R	R	R	R
C3: Unchanged-document control[b]	A	A	A	A

[a]These three changes affect fields omitted from the context key. Context admission produces semantic-violating writes under both executors; content and instance binding reject them.
[b]The reviewed and submitted request values both remain 101.00, so every binding admits. The changed document supplies 999.00 to unchecked execution, which writes it; verified execution blocks the write. The unchanged-document control writes 101.00. This request-value guard is not the full contract's P3 check on the committed value, which would reject 999.00.
Raw content omits money normalization and is diagnostic. A note is compared when the case declares it semantic.

6 Paired Agent Execution

Once a candidate is rejected, the agent can reuse the reviewed instance, obtain matching authorization, or stop when no route remains. Paired continuations make each choice observable and record the interactions it requires.

6.1 Design and measurement

Both policies run from the same review checkpoint, so the instance conjunct is the only difference between paired arms. The original study freezes 128 tasks across two provider configurations and two policies: 512 planned arms, one realization per cell. The six families are legal continuation (L, 32), refreshed preview (N, 32), queued handoff (G, 16), changed evidence (E, 16), stale predecessor (V, 16) and receipt unavailability (R, 16). In both collections, all families share an initial instruction not to treat a preview candidate as automatically authorized; N additionally states that a preview does not itself authorize a commit. A deterministic reviewer creates a shared authorization; any candidate that passes validity checking is granted immediately, and a fresh authorization supersedes earlier grants. The conversation and persistence checkpoint are then copied into isolated context and bound arms with balanced policy order (Figure 3).

At the queued handoff (G), the harness submits a standardized context-equivalent substitute, after which actual gate feedback enters the next agent decision. The refreshed-preview family (N) leaves management of a required refreshed preview to the agent. Receipt unavailability (R) makes the first receipt query unavailable after a

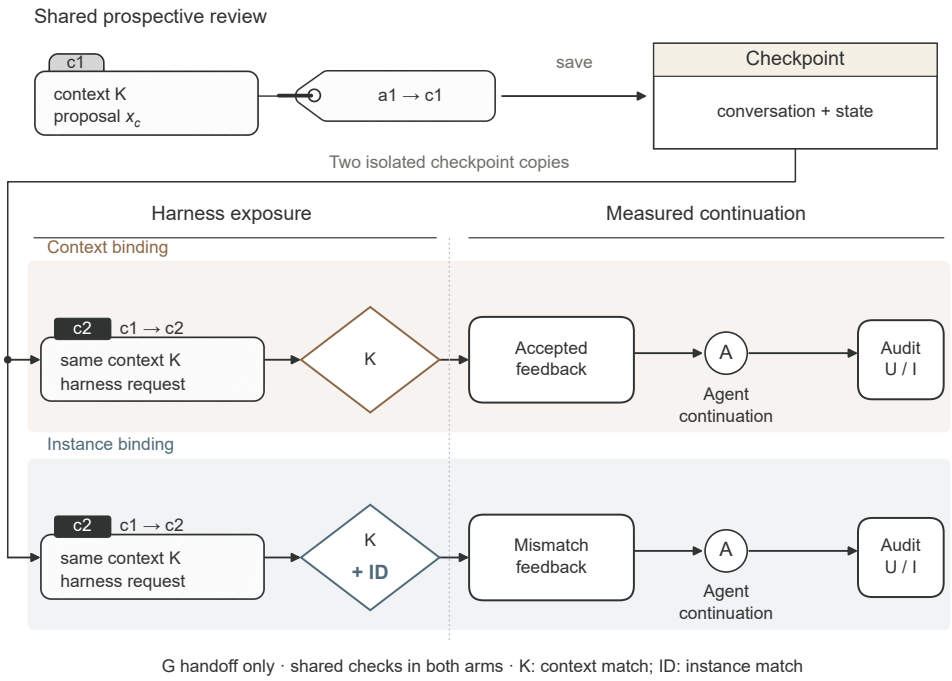


Fig. 3 At standardized G handoffs, paired gates differ only in instance equality. The reviewed candidate c1 and its authorization a1 are saved with the conversation and state, then copied into two isolated, directly labelled policy arms. The harness submits c2 with the same context and a different identifier. K denotes K_{eq}; the instance gate adds ID, denoting I_{eq}, while all other checks are shared. Both admission and rejection produce actual feedback for the next sampled agent decision (A), followed by an independent U/I audit. A rejected request makes no commit but leaves the continuation active; subsequent requests must still pass their arm's gate. Measured G outcomes cover 32 delivered pairs from 16 task definitions. N leaves preview management to the agent and is not depicted. Object glyphs were AI-generated; see the AI-assisted preparation declaration

successful commit response has already been delivered; it covers only this confirmed-commit path, and unknown-outcome recovery is a separate deterministic probe (Online Resource 2, A3).

The design separates exposure from behavior. At G, the harness creates and submits the substitute; context admission and the resulting $I = 0$ follow from the policy and oracle definitions, so the 17 and 15 substitution counts are exposure counts rather than agent actions. The harness has already executed the context-side commit when continuation begins; the bound side must recover from rejection. The cost contrast includes this required work. Behavioral observations begin with delivered feedback: whether context arms complete, which legal route bound arms choose, their action sequence through the first persistent recovery commit, and any subsequent confirmation calls.

The online host is a separate reduced implementation shared by A and B, not the relational gate of Section 5. Tasks commit one field and record its source as the successor

transition identifier, exercising reduced P5/P6. Controlled implementations test multi-field successors, deep source chains and the full evidence graph. With deterministic grants for valid candidates, reauthorization measures agent routing rather than human review burden.

Configuration A requests `gpt-5.6-terra` through Codex CLI 0.160.1 with low reasoning; no returned model identifier is exposed. B requests and receives `deepseek-v4-flash` through an Anthropic-compatible Messages API at temperature zero. Both use the same execution system. Each suffix permits 20 decisions and 24 tool calls, with a 90-second request timeout and a 720-second suffix limit. One format-repair allowance applies across the full prefix-plus-suffix history. B has a 4,096-token response cap; A's upstream cap is unavailable. Frozen inputs and complete settings are in Online Resource 1.

Collection provenance. A provider quota prevented 117 original A pairs from receiving any model response before review. Those previously unexposed pairs form a separately frozen supplementary A collection of 234 arms, retaining task definitions, prompts, tools, policy ordering, requested configuration and scoring. Original outcomes remain unchanged; the later, selected collection has its own denominator and exploratory analysis.

The independent oracle scores utility U from authoritative goal attainment, required workflow and accurate final reporting. Integrity I is 1 with sufficient coverage and no contract violation. A proved violation yields 0, and incomplete necessary evidence yields unknown unless a violation is already proved. With no authoritative transition, integrity can hold vacuously and carries no evidence about contract conformance. The instance-continuity oracle applies to both policies, so a context-admissible substitution can score $I = 0$: $I = 0$ records non-conformance to the declared instance rule, not a safety failure under the context rule that the arm declares. Admissibility under a policy and conformity to the independent oracle are different judgments.

For aggregate completion differences, each task contributes the mean of its configuration-specific paired differences (bound minus context). We bootstrap 128 task clusters within family, retaining both policies and configurations (10,000 iterations, seed 20261007). We apply the same procedure separately to the 117 supplementary pairs. Recovery summaries condition on feedback in a saved model input followed by a received response. Recovery calls run from that feedback through the first successful commit that adds a persistent transition; subsequent calls are counted separately. Complete-suffix cost remains the bound-minus-context difference over both full continuations.

6.2 Task completion and instance continuity

Substitutions that context binding admits mostly complete their tasks. All proved substitutions occur at G under context binding: 17 in the original collection and 15 in supplementary A. Thirty of these 32 arms still complete their task, so an audit of endpoint utility alone would classify them as successful; the continuity oracle separates them. The 32 episodes come from 16 G task definitions, each observed under deployments B and A. The corresponding bound arms reject the substituted request and all finish with $U = 1, I = 1$. Table 6 retains complete cohort denominators,

Table 6 Completion and instance substitution by collection. Context (Ctx) and instance (Inst; archived as bound) counts are separate; completion denominators remain the planned pairs.

Collection	Planned pairs	Verified checkpoints	Completion		Substitutions		I unknown	
			Ctx	Inst	Ctx	Inst	Ctx	Inst
Original A	128	11	9	9	1	0	0	0
Original B	128	128[a]	117	119	16	0	1	1
Supplementary A	117	117	114	116	15	0	0	0

Planned pairs are also planned arms per policy; both policies together double that count. Checkpoints count verified prefix events and matching saved states. Original A includes 117 pairs stopped by quota before review; supplementary A evaluates those pairs later. [a]All 128 original-B pairs reached review, but only 127 pair-summary flags survive. N-020-B has a saved checkpoint event and state; its context suffix is partial and its bound suffix was not started. These two integrity scores remain unknown. I denotes instance-rule conformance; no-transition arms can satisfy it vacuously.

including quota failures and unknown integrity; family-level joint outcomes are in Online Resource 1, S11–S12.

Planned-ledger completion includes pre-review service failures. Original B completes 117/128 context and 119/128 bound arms (+1.56 percentage points; 95% task-cluster interval [−3.91, 7.03]). Original A completes 9/128 under each policy with no discordance. The pooled 126/256 versus 128/256 contrast and interval are therefore exactly half the B values: +0.78 points [−1.95, 3.52]. This is a denominator rescaling of the same discordance, not additional precision. The supplementary difference is +1.71 points [0.00, 4.27]. Neither comparison establishes a completion advantage (Section 7.2). Original B has 18 capped invalid-output terminations (10 context, 8 bound), including two G context arms; this imbalance constrains interpretation of the original-B completion difference.

A later 24-task output-budget check gives context/bound completion of 23/24 versus 22/24 at 4,096 tokens, and 24/24 for both at 393,216 tokens. Integrity remains 20/24 versus 24/24 at both caps, including four programmed context substitutions per cap. This 96-continuation study uses a later collection period and longer waits; its full results and the original failure decomposition are in Online Resource 2, A1–A2.

6.3 Two routes to recovery and their cost

All 32 delivered G-bound mismatches recover. Original B's 16 continuations reuse the reviewed candidate; supplementary A has eight reuse and seven reauthorization paths; original A's single mismatch also uses reauthorization. Agents thus select different legal continuations under the same admission rule (Figure 4).

A post-hoc reconstruction checks whether these choices reflect different available routes. For all 32 episodes, the saved rejection state retains an active original grant, an unchanged reviewed candidate, valid evidence and a current predecessor, and the saved model input contains that candidate, grant and authorized value. These checks

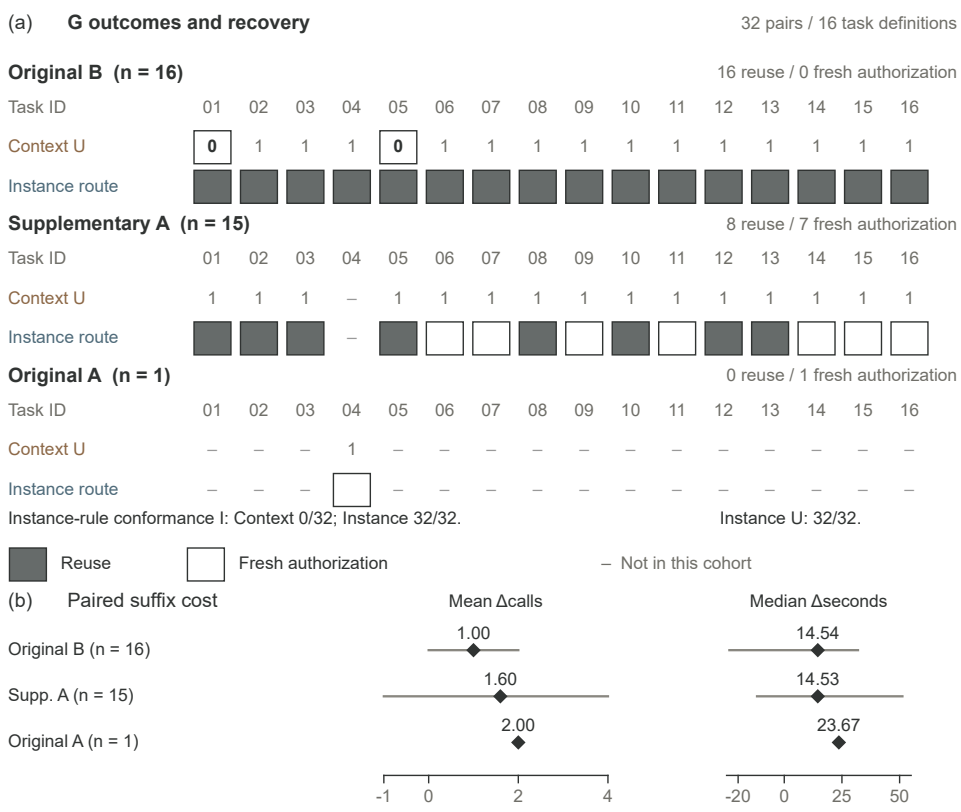


Fig. 4 All 32 delivered G pairs across 16 task definitions. (a) Task columns align across cohorts; dashes mean absent from that cohort's delivered pairs, not failure. Supplementary A omits 04; Original A includes only 04. I denotes instance-rule conformance, not safety under the context policy. Context has $I = 0$ throughout and $U = 1$ in 30 pairs; all bound arms end with $U = 1, I = 1$. Filled/open squares denote reuse/fresh authorization. (b) Diamonds show mean call and median elapsed-time differences for complete paired suffixes (Δ = bound minus context); segments show observed minima–maxima, not confidence intervals. Time includes decisions, tool interactions and terminal paths. Table 7 separates recovery calls

use the state and input bytes rather than the host's logged list of legal paths. Reuse was therefore both available and visible at the observed boundary, including in all eight episodes that requested fresh authorization.

After delivered feedback, each of the 24 reuse episodes makes one commit; each of the eight fresh-authorization episodes requests authorization and then commits. No failed or exploratory call intervenes. Later receipt or state queries are separate continuation actions (Table 7); grants are immediate and deterministic. Across the 16 matched task definitions, A and B both reuse in eight; A reauthorizes while B reuses in eight. A combines one original and fifteen later supplementary episodes, so this is a descriptive route contrast, not an identified model effect (Online Resource 2, A12). Online Resource 1, S12.6 illustrates the saved input, recovery commit and subsequent receipt query in one episode.

Table 7 Agent tool calls at delivered G handoffs. All entries except n are cohort means; post-commit calls occur after the first successful persistent recovery commit.

Collection	n	To recovery commit	After commit	Bound suffix	Context suffix	Δ
Original A	1	2.000	2.000	4.000	2.000	2.000
Original B	16	1.000	0.375	1.375	0.375	1.000
Supplementary A	15	1.467	1.533	3.000	1.400	1.600

Recovery comprises one commit in each of 24 reuse episodes and an authorization request followed by a commit in each of eight fresh-authorization episodes. The last column is bound minus context over complete suffixes, not calls beyond a route minimum. Values are recomputed from the archived event order and persistent transitions; no new runs are added.

Complete paired suffixes add a mean 1.00 tool call and median 14.54 seconds in original B ($n = 16$ pairs), versus 1.60 calls and 14.53 seconds in supplementary A ($n = 15$). A new exploratory paired bootstrap gives 95% intervals of [0.75, 1.25] and [0.87, 2.33] calls, respectively; these condition on the constructed task sample (Online Resource 2, A12). The latter's seven reauthorization paths add 0.47 authorization requests per pair; original B adds none. Original A's single pair adds two calls and 23.67 seconds, including one authorization request; no interval is estimated. Full-suffix differences include required recovery, later confirmation and context-side calls; they need not equal route minima. These request counts expose a cost that immediate deterministic grants cannot price: a real workflow may require another review and queue wait. They are not estimates of human time. Table 7 separates recovery, subsequent and context-side calls; Figure 4 preserves signed ranges. E/V/R results remain separate in Online Resource 1, S11–S12.

6.4 Recovery when permission or feedback changes

Recovery also depends on current permission, not just candidate identity. A separately frozen exploratory study collected on 14 October 2026 uses four task instances from the same G family, each under three post-rejection states: the old grant remains valid, it is revoked while the reviewer remains available, or the reviewer is unavailable and no legal route remains. Each state compares current permission facts alone with the same facts plus derived legal-path labels. The resulting 12 pairs comprise 24 hosted trajectories using deployment B's requested alias, a 393,216-token output cap and longer waits; 24 deterministic rule trajectories are separate controls. Each cell runs once. Online Resource 2, A13 gives selection, allocation, raw-record audit and reproduction details.

Both feedback variants complete all eight feasible episodes and stop without a write in all four unavailable episodes; all twelve episodes per variant preserve integrity. Each variant uses sixteen tool calls, including four authorization requests, with zero rejected continuation operations. Paired call differences (labels minus facts) are -1 in three pairs, zero in six and $+1$ in three. The rule controller also completes feasible episodes and stops in unavailable ones. By permission state, the four paired call differences sum to $+2$ for valid grants, $+1$ for revoked grants and -3 when no route remains; the

aggregate zero masks this descriptive variation. Secondary totals are lower with labels: 283.751 versus 371.048 seconds and 49,293 versus 66,774 output tokens; A13 retains all state-specific results. These twelve pairs reuse four task definitions. The study shows permission-sensitive continuation but no net tool-call benefit from path labels. It neither establishes feedback equivalence nor supports an LLM-specific advantage. The sample is kept separate from the original completion estimates and the 32 delivered G pairs.

A new dynamic pilot separates availability at feedback from permission at the next commit. Two reused tasks cross four conditions: no change, selected-grant revocation, predecessor advance, and no remaining legal route. All eight hosted first commits were initially admissible; six were correctly rejected after the injected change. Hosted and adaptive-rule controllers both restore all six feasible business states and correctly stop in the two unavailable cases. A cached-route control restores only the two static cases. Independent SQLite readback verifies the effects and final reports. This pilot does not establish an LLM advantage; Online Resource 2, A16 preserves all 24 trajectories and a narrower frozen confirmation metric requiring a matching final `get_state` query.

6.5 Exposure without rejection feedback

Agents shown a context-equivalent alternative reused the reviewed instance or obtained fresh authorization. The N family supplies such an alternative without forcing its submission. Across the collections, 186 arms were planned, 185 retain state and 127 reached review. Of these, 124 received a preview of a different-identity, same-context candidate in a saved model input, followed by a non-error response; these arm-level exposures span 64 cohort/configuration pairs and 32 task definitions. Sixty pairs expose both arms and four expose one. Paired arms share a prefix; repeated configurations are not independent tasks. Agents reused the reviewed instance in 123 cases and obtained matching authorization for the previewed candidate in one; there were zero unauthorized substitutions.

One original-B context arm supports only observation through its saved after-state, and its bound counterpart was not started; both integrity scores remain unknown. Excluding the partial arm leaves 126 reviewed arms, 123 exposures, 122 reuses and one reauthorization (Online Resource 2, A7). N thus shows two continuity-preserving behaviors when an alternative is available. G measures recovery from delivered mismatch feedback; N contains no such feedback.

7 Discussion

7.1 Implications

The evidence is a comparison of scopes and their continuations, not a new enforcement mechanism. Omitted fields, equivalent replacements and mutable referents require different checks; persisted-state and extension tests show where those checks can run. Admission alone does not determine continuation: eight recovered mismatches prompted fresh authorization despite an available, visible grant.

A context-scoped policy that admits an identity-different, content-identical candidate is following its own rule. The continuity oracle records non-conformance to the instance rule, not a safety failure, and the engineering question is which scope an application declares and whether its continuation interface is consistent with that choice.

Because the candidate, grant and authorized value were present in the saved model input, missing information within this fixture cannot explain the eight fresh authorizations. The agent's interpretation of rejection and the offered continuation remain relevant design factors that an admission-only audit misses.

Endpoint-value audits cannot distinguish reviewed artifacts from regenerated equivalents. Approval scope instead requires auditing the relation among candidate, authorization and committed transition.

The exploratory feedback comparison sharpens the interface implication. With the same permission facts, adding path labels produces no net tool-call reduction; when no legal route remains, truthful stopping is the appropriate endpoint. An implementation should expose current authority and the reviewed object, then distinguish recovery, confirmation and renewed review in its telemetry. The evidence does not prescribe a universal route-selection strategy.

Read together, the results make approval scope, the enforced execution boundary and recovery behavior one interdependent design problem, decided at four points:

1. **Obtain the responsibility rule.** Have the workflow owner state whether approval covers a value, retained context, declared content or the reviewed artifact, and which replacements may reuse it. The measurements do not decide this organizational requirement.
2. **Cover the reusable content.** Put each decision-relevant field in the reuse key or check its approved value separately at execution; declare the comparator.
3. **Anchor binding and recovery.** Resolve identifiers through trusted lookup to immutable candidates; return the reviewed identifier or the state/evidence mismatch needed for reuse or fresh authorization. A valid original grant can support reuse even for an irreversible operation; stale state or evidence can require fresh authorization even for a reversible one.
4. **Check external objects at consumption.** Verify the reviewed digest and execute on the same captured bytes, as in the binding-scope experiment's verified executor.

ToolGate callbacks and a commit bridge carry the application obligations; the native-to-store experiment checks their composition with interrupt routing, stale state and rollback. Deployment still requires restart-safe grant/checkpoint storage, authenticated review and evaluation with autonomous agents and real approval delays.

7.2 Threats to validity

Construct validity. Measured costs are agent-side tool calls and suffix time. Gate computation was not isolated; human re-inspection and approval-queue waiting were not measured. The deterministic reviewer immediately grants any valid request, whereas a deployed reauthorization request may wait, be refused or require transfer to an authorized reviewer. The observed routes and suffix times do not establish optimal

route selection or production approval latency. Canonical equality is enforced at the gate; agreement between displayed and compared values was tested only in the browser fixture. The binding-scope study implements the selected comparators and a common executor, not the complete set of shared checks G, and normalizes money to two decimals instead of canonical JSON equality.

Internal validity. G substitutions are harness-created exposures (Section 6.1), and each cell has one realization. G recovery and N continuation were observed under the common preview–authorization instruction, with an additional reminder in N; no control without warnings isolates these prompts' effects. The archival availability audit is post-hoc and establishes a saved-state property, not the outcome of an unchosen counterfactual route. The completion comparisons are not powered for a non-inferiority claim, and an interval that includes zero does not establish equivalence. The supplementary cohort and output-budget study retain separate collection conditions and denominators. The feedback comparison uses four previously used task definitions in a separately frozen collection; its paired fact control isolates added labels within that fixture, but not effects of all feedback or differences across collection periods. The deterministic binding protocol was amended after inspecting an initial run, and its 72 routes are crossed conditions over nine constructed cases, not independent samples.

External validity. Constructed tasks, deterministic review and the reduced single-field host leave natural mismatch prevalence, human approval cost and production benefit unmeasured. ToolGate uses scripted decisions and an author-written commit bridge, not native default enforcement or autonomous agents. Native-to-store checkpoints and pre-commit grant handles remain in memory. The separate SDK checks cover clean restarts, continuation routes and controlled substitution of supplied history; they do not test crash recovery, restart-safe application grants, concurrent replay, reviewer authentication or other hostile pending-state mutations. Each SDK result is limited to its pinned version and tested route, not framework-wide safety. Only two design-space points have agent evidence; the remaining combinations lack developer-study validation.

Measurement boundary. The native checks compare executed effects keyed by the runtime tool-call identifier written by the tool body. Recovering the reviewed-call mapping when that identifier itself cannot be trusted is a separate open problem and is not addressed here.

Scope of the formal results. The formal and persistence results apply to the stated local observation model and atomic single-record successor (Section 3.3); multi-record or distributed enforcement requires additional coordination.

7.3 Directions for future work

Four directions follow from the boundaries above. Recovering the reviewed-call mapping when the runtime call identifier cannot be trusted is a separate open problem: it needs either a trustworthy execution identifier or an independent record of what the tool body executed. Restart-safe and concurrently enforced authorization state, reviewer authentication and hostile mutation of pending state are untested here, and each matters once approval leaves a single in-process host. Replacing the deterministic reviewer with a production approval queue would measure human approval latency

and the real cost of renewed authorization. Finally, only two points of the design space have agent evidence; developer studies of the remaining combinations, and of multi-record or distributed enforcement, would show how far the contract extends.

8 Conclusion

An approval's binding object determines which replacements it permits. That is an admission decision; how an agent recovers after a rejection is a separate, observable choice. The four-role correction contract connects the admission choice to authorized values, successor structure and field sources. The tested agents recover from all delivered identity-only rejections when the original grant remains usable and present in model input, yet sometimes request new authorization. The exploratory permission-state study also demonstrates stopping when no legal route remains, with no net tool-call reduction from additional path labels in this sample. The practical implication is to enforce a declared approval scope and to measure its continuation, including authorization requests whose human-review costs remain unmeasured.

Statements and Declarations

Data and software availability

Constructed tasks and archived review checkpoints underpin the study. Frozen archives retain the controlled experiments at commit 2645e5e18c900ea91c9c980e44195dc71e410432, and the reference implementation, formal models and earlier historical evidence at c6d512843c905cab6d8521dd8c914f7fb26d85ae. The prospective inputs and runner have master-manifest SHA-256 7773563b40c8db30d6def121915f50fbad032537edde32afd4ba39ec3eea744b. The handoff archive at 475e66f549613aaaaa62c2ec81ffdee483636c54 contains the complete original ledger, responses, states, scorer, tables, figure sources and hashes (Online Resource 1). Supplementary A retains its separate freeze, complete attempts, independent scores and source hashes, with the offline entry point in S12. The pinned reproduction guide orders the offline workflow. Reanalysis needs no model calls; new hosted runs require provider access.

`X1-X2-reproduction.zip` contains versioned sources, two saved executions and independent offline SQLite verification for native approval and binding scope (Online Resource 2, A11). A17 documents three separate archives: `Correction-consumption-reproduction.zip` (504 cells), `Agno-entrypoint-reproduction.zip` (54) and `Reviewed-instance-substitution-reproduction.zip` (16). These retain frozen protocols, code, cases, source/SDK seals, raw per-cell effects, scores, reports and portable offline verifiers; the first also retains adapter and scoring reviews.

`Native-contract-reproduction.zip` retains the scripted native-to-store source freeze, unchanged backend dependencies, two saved runs, independent SQL scorer and negative controls (Online Resource 2, A14); it calls no hosted model.

The source package supplies recovery-cost readback and hashes, an anchored example, availability reconstruction for all 32 G pairs, and Table 7 recomputation. `REPRODUCE-RECOVERY.md` lists the two archive dependencies, hashes and offline commands; `-output` separates derived files. These analyses add no runs.

`Feedback-sensitivity-reproduction.zip` supplies the frozen exploratory study, raw trajectories, dependencies and read-only instructions in `REPRODUCE.md`. `Dynamic-recovery-reproduction.zip` contains the pilot protocol, eight hosted trajectories, sixteen rule controls, independent scoring and offline replay. Original and supplementary ledgers remain separate.

Ethics approval and consent to participate

The prospective experiment uses constructed tasks and a deterministic host reviewer, with no new human participants. In the historical annotation study, one author (X.W.) and one non-author volunteer independently coded benign completion outcomes; L.H. adjudicated their single inter-coder disagreement and separately reviewed eleven cases in which the strict trajectory label and the endpoint rule disagreed; a separate textual-acknowledgment audit with 39 human–rule disagreements was not adjudicated and is reported only as human–rule agreement. X.W. also completed the textual-acknowledgment annotation. Only pseudonymous annotator identifiers were collected. Both coders were informed of the task and data use, and the non-author coder participated voluntarily. The authors assessed the annotation of generated outputs as not requiring ethics approval. Online Resource 1, S5.2 details the annotation procedure. No animal subjects were involved.

Consent for publication

Not applicable; no identifiable participant information is reported.

CRediT authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing–original draft, and Writing–review & editing. Xuan Wentao: Methodology, Formal analysis, Investigation, Data curation, Validation, Visualization, and Writing–review & editing. Chen Qile: Investigation and Validation. Peng Peng: Supervision and Writing–review & editing.

Funding and competing interests

This research received no external funding. The authors declare no competing interests.

AI-assisted preparation

ChatGPT (GPT-6 Astra) was used for language polishing and selected experimental verification tasks. Codex was used solely to audit and review experimental procedures and results.

The illustrative object glyphs in Figures 1–3 were generated on 21 October 2026 using OpenAI's built-in image-generation tool from author-defined specifications. The diagrams were assembled in code, with labels, values, and connections specified separately. The image-generation model version was not provided by the interface. The figure source files retain the specifications, original assets, and placement code.

The authors retain full responsibility for the research design, analysis, interpretation, and final manuscript.

Supplementary information

Online Resource 1 contains formal checks, workloads, historical records, prospective definitions, complete outcomes and the artifact map. Online Resource 2 contains supplementary experiments and audits (A1–A8), extended specification and design guidance (A9–A10), native approval and binding scope (A11), recovery availability and feedback (A12–A13), native-to-store composition (A14), obligation foundations (A15), dynamic recovery (A16) and the three native-SDK checks (A17). Online Resource 3 collects the editable sources and seven reproduction archives described above. Original and supplementary ledgers remain separate.

References

Arab BS, Gawlick D, Krishnaswamy V, et al (2018) Using reenactment to retroactively capture provenance for transactions. IEEE Transactions on Knowledge and Data Engineering 30(3):599–612. https://doi.org/10.1109/TKDE.2017.2769056

Bacchelli A, Bird C (2013) Expectations, outcomes, and challenges of modern code review. In: 2013 35th International Conference on Software Engineering (ICSE), pp 712–721, https://doi.org/10.1109/ICSE.2013.6606617

Barres V, Dong H, Ray S, et al (2026) τ^2-Bench: Evaluating conversational agents in a dual-control environment. In: Proceedings of the 43rd International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 306. PMLR, pp 6736–6782, URL https://proceedings.mlr.press/v306/barres26a.html

Bindschaedler L, Kossmann F, Liu C, et al (2026) Guarded commits: Transactional human approvals for LLM workflows. https://doi.org/10.48550/arXiv.2610.00037, URL https://arxiv.org/abs/2610.00037, preprint, version 1, arXiv:2610.00037

Buneman P, Khanna S, Tan WC (2001) Why and where: A characterization of data provenance. In: Database Theory—ICDT 2001, Lecture Notes in Computer Science, vol 1973. Springer, pp 316–330, https://doi.org/10.1007/3-540-44503-X_20

Buneman P, Chapman A, Cheney J (2006) Provenance management in curated databases. In: Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data, pp 539–550, https://doi.org/10.1145/1142473.1142534

Chen Z, Liu H, Xu D, et al (2026) Cordon: Semantic transactions for tool-using LLM agents. https://doi.org/10.48550/arXiv.2606.17573, URL https://arxiv.org/abs/2606.17573v1, arXiv preprint, version 1, arXiv:2606.17573

Cheney J, Chiticariu L, Tan WC (2009) Provenance in databases: Why, how, and where. Foundations and Trends in Databases 1(4):379–474. https://doi.org/10.1561/

1900000006

Clark DD, Wilson DR (1987) A comparison of commercial and military computer security policies. In: 1987 IEEE Symposium on Security and Privacy, pp 184–194, https://doi.org/10.1109/SP.1987.10001

Gerrit Project (2026a) Gerrit code review: Review labels. URL https://gerrit-review.googlesource.com/Documentation/config-labels.html, official documentation, copyCondition and Sticky Votes. Accessed 21 October 2026

Gerrit Project (2026b) Gerrit code review: Submit requirements. URL https://gerrit-review.googlesource.com/Documentation/config-submit-requirements.html, official documentation. Accessed 21 October 2026

GitLab (2026a) Deployment approvals and deployments API: approve or reject a deployment. URL https://docs.gitlab.com/api/deployments/, official documentation. The Deployments API records approval per deployment through `POST /projects/:id/deployments/:deployment_id/approval` with status `approved` or `rejected`, and one user may approve a given deployment only once; protected-environment rules are described at https://docs.gitlab.com/ci/environments/deployment_approvals/. Accessed 24 October 2026

GitLab (2026b) Merge request approval settings. URL https://docs.gitlab.com/user/project/merge_requests/approvals/settings/, official documentation. Accessed 21 October 2026

Gray J (1981) The transaction concept: Virtues and limitations. In: Proceedings of the Seventh International Conference on Very Large Data Bases, pp 144–154, URL https://www.vldb.org/conf/1981/P144.PDF

Haeberlen A, Kouznetsov P, Druschel P (2007) PeerReview: Practical accountability for distributed systems. In: Proceedings of the Twenty-First ACM SIGOPS Symposium on Operating Systems Principles, pp 175–188, https://doi.org/10.1145/1294261.1294279

Hardy N (1988) The confused deputy: (or why capabilities might have been invented). ACM SIGOPS Operating Systems Review 22(4):36–38. https://doi.org/10.1145/54289.871709

He J, Yu D (2026) Beyond memory: A transactional continuity kernel for long-lived AI agents. https://doi.org/10.48550/arXiv.2608.11632, URL https://arxiv.org/abs/2608.11632v1, arXiv preprint, version 1, arXiv:2608.11632

He J, Veltri E, Santoro D, et al (2016) Interactive and deterministic data cleaning. In: Proceedings of the 2016 International Conference on Management of Data, pp 893–907, https://doi.org/10.1145/2882903.2915242

Jackson D (2002) Alloy: A lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11(2):256–290. https://doi.org/10.1145/505145.505149

Kumar AA (2026) Loopjacking: Hijacking human-in-the-loop approval. https://doi.org/10.48550/arXiv.2609.21081, URL https://arxiv.org/abs/2609.21081, preprint, version 1, arXiv:2609.21081

Kung HT, Robinson JT (1981) On optimistic methods for concurrency control. ACM Transactions on Database Systems 6(2):213–226. https://doi.org/10.1145/319566.319567

LangChain (2026) Human-in-the-loop. URL https://docs.langchain.com/oss/python/langchain/human-in-the-loop, official documentation. Accessed 21 October 2026

Leucker M, Schallhart C (2009) A brief account of runtime verification. The Journal of Logic and Algebraic Programming 78(5):293–303. https://doi.org/10.1016/j.jlap.2008.08.004

Liu Y, Peng X, Cao J, et al (2026) ToolGate: Contract-grounded and verified tool execution for LLMs. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, pp 9653–9684, https://doi.org/10.18653/v1/2026.findings-acl.470, URL https://aclanthology.org/2026.findings-acl.470/

Lodderstedt T, Richer J, Campbell B (2023) OAuth 2.0 rich authorization requests. RFC 9396, RFC Editor, https://doi.org/10.17487/RFC9396, URL https://www.rfc-editor.org/rfc/rfc9396.html

Long M, Zhao Y, Wang H (2026) LogicHunter: Testing LLM agent frameworks with an agentic oracle. In: Proceedings of the ACM on Software Engineering (ISSTA 2026), https://doi.org/10.1145/3832142, arXiv:2607.06195

Microsoft (2026) Using function tools with human in the loop approvals. https://learn.microsoft.com/en-us/agent-framework/agents/tools/tool-approval, microsoft Agent Framework documentation, last updated 2026-10-15; accessed 2026-10-24

Miller MS (2006) Robust composition: Towards a unified approach to access control and concurrency control. PhD thesis, Johns Hopkins University, Baltimore, Maryland, URL https://hdl.handle.net/1774.2/873

OpenTofu (2026) Command: apply. URL https://opentofu.org/docs/cli/commands/apply/, official documentation, Saved Plan Mode. Accessed 21 October 2026

Pydantic Services Inc. (2026) Pydantic AI documentation: messages and chat history. https://pydantic.dev/docs/ai/core-concepts/message-history/, accessed 2026-10-24.

Documents the trust boundary for client-supplied message history

Sabelfeld A, Myers AC (2003) Language-based information-flow security. IEEE Journal on Selected Areas in Communications 21(1):5–19. https://doi.org/10.1109/JSAC.2002.806121

Santos-Grueiro I (2026) Temporary authority, permanent effects: Commit-time authorization for LLM agents. https://doi.org/10.48550/arXiv.2607.10487, URL https://arxiv.org/abs/2607.10487v1, arXiv preprint, version 1, arXiv:2607.10487

Schneider FB (2000) Enforceable security policies. ACM Transactions on Information and System Security 3(1):30–50. https://doi.org/10.1145/353323.353382

Torlak E, Jackson D (2007) Kodkod: A relational model finder. In: Tools and Algorithms for the Construction and Analysis of Systems, Lecture Notes in Computer Science, vol 4424. Springer, pp 632–647, https://doi.org/10.1007/978-3-540-71209-1_49

Wang Q, Wang C, Meng J, et al (2026a) Authorization closure graph: Minimal repair for llm agents with evolving user instructions. URL https://arxiv.org/abs/2609.32428v1, preprint, version 1, arXiv:2609.32428

Wang Y (2026) Approval laundering: Systematizing approval–execution binding failures in AI coding-agent harnesses. https://doi.org/10.48550/arXiv.2609.38983, URL https://arxiv.org/abs/2609.38983, preprint, version 1, arXiv:2609.38983

Wang Z, Li J, Guo X, et al (2026b) From noisy feedback to evidence-aware issue specifications: an agent-governed retrieval-augmented generation approach. Automated Software Engineering 33:103. https://doi.org/10.1007/s10515-026-00640-x

World Wide Web Consortium (2013) PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, URL https://www.w3.org/TR/2013/REC-prov-dm-20130430/, edited by Luc Moreau and Paolo Missier. Recommendation dated 30 April 2013

Yakout M, Elmagarmid AK, Neville J, et al (2011) Guided data repair. Proceedings of the VLDB Endowment 4(5):279–289. https://doi.org/10.14778/1952376.1952378

Zhu J, Liu Z, Fan S, et al (2026) From approval to execution: Reconstruction-aware repair analysis for LLM-agent software. https://doi.org/10.48550/arXiv.2609.26529, URL https://arxiv.org/abs/2609.26529, preprint, version 1, arXiv:2609.26529